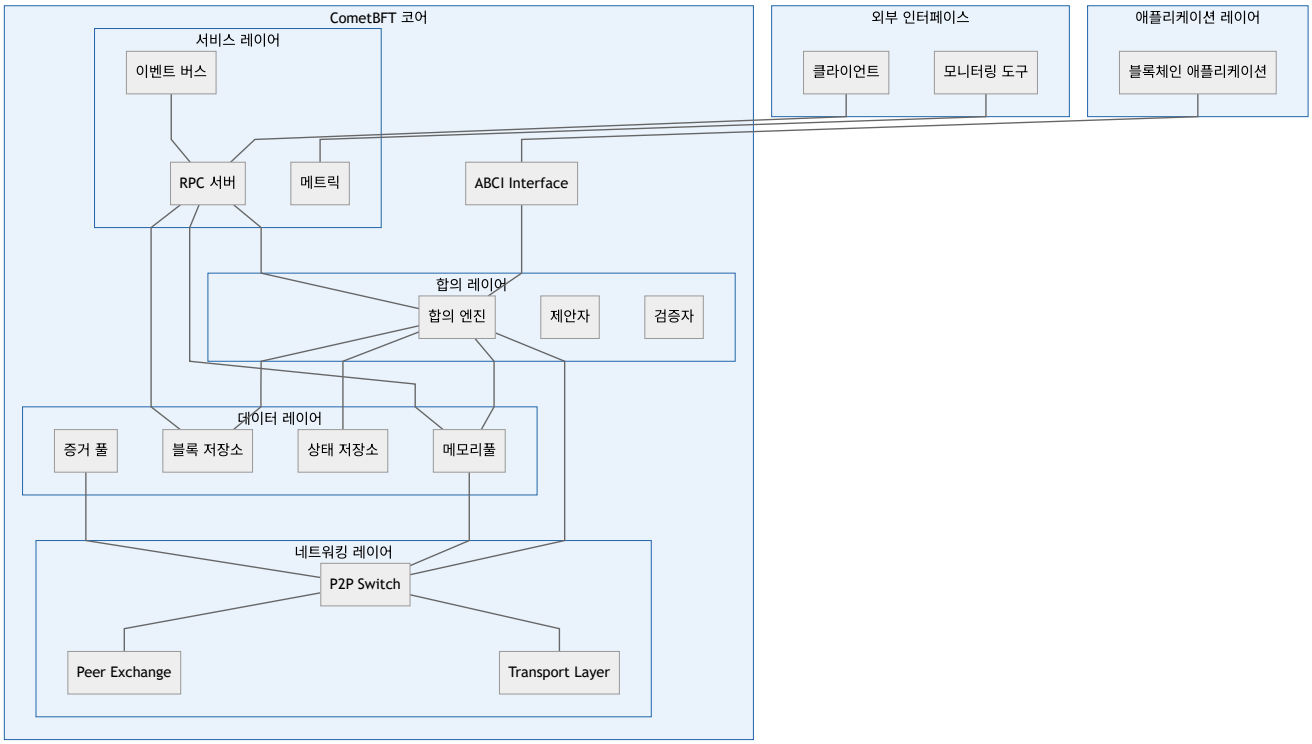
CometBFT 전체 아키텍처

개요

CometBFT는 모듈러 아키텍처를 채택하여 각 컴포넌트가 명확한 책임을 가지고 상호작용하는 분산 합의 시스템입니다. 이 문서는 CometBFT의 전체 시스템 구조와 주요 컴포넌트들 간의 관계를 상세히 설명합니다.

시스템 전체 구조도



주요 컴포넌트 상세 분석

1. ABCI (Application Blockchain Interface)

ABCI는 CometBFT의 핵심 혁신 중 하나로, 애플리케이션과 합의 엔진을 분리하는 인터페이스입니다.

주요 특징:

- **언어 독립성**: 애플리케이션을 어떤 언어로든 개발 가능
- **명확한 생명주기**: 트랜잭션 처리의 각 단계가 명확히 정의됨
- **상태 분리**: 애플리케이션 상태와 합의 상태의 명확한 구분

핵심 메서드:

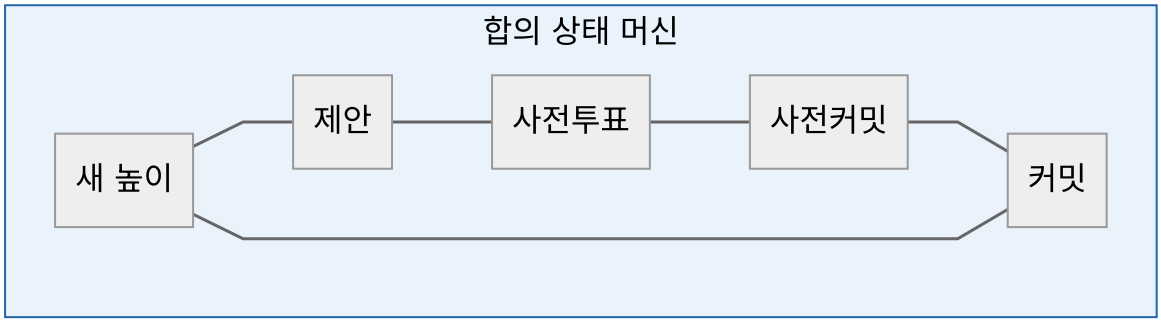
```
type Application interface {
// 정보 조회
Info(ctx context.Context, req *InfoRequest) (*InfoResponse, error)
Query(ctx context.Context, req *QueryRequest) (*QueryResponse, error)
// 메모리풀 연결
CheckTx(ctx context.Context, req *CheckTxRequest) (*CheckTxResponse, error)
// 합의 연결
InitChain(ctx context.Context, req *InitChainRequest) (*InitChainResponse, error
PrepareProposal(ctx context.Context, req *PrepareProposalRequest) (*PrepareProp
ProcessProposal(ctx context.Context, req *ProcessProposalRequest) (*ProcessProp
FinalizeBlock(ctx context.Context, req *FinalizeBlockRequest) (*FinalizeBlockRes
Commit(ctx context.Context, req *CommitRequest) (*CommitResponse, error)
// 상태 동기화
ListSnapshots(ctx context.Context, req *ListSnapshotsRequest) (*ListSnapshotsRes
OfferSnapshot(ctx context.Context, req *OfferSnapshotRequest) (*OfferSnapshotRes
```

```
}
```

2. 합의 엔진 (Consensus Engine)

Tendermint 합의 알고리즘을 구현하는 핵심 컴포넌트입니다.

구조:



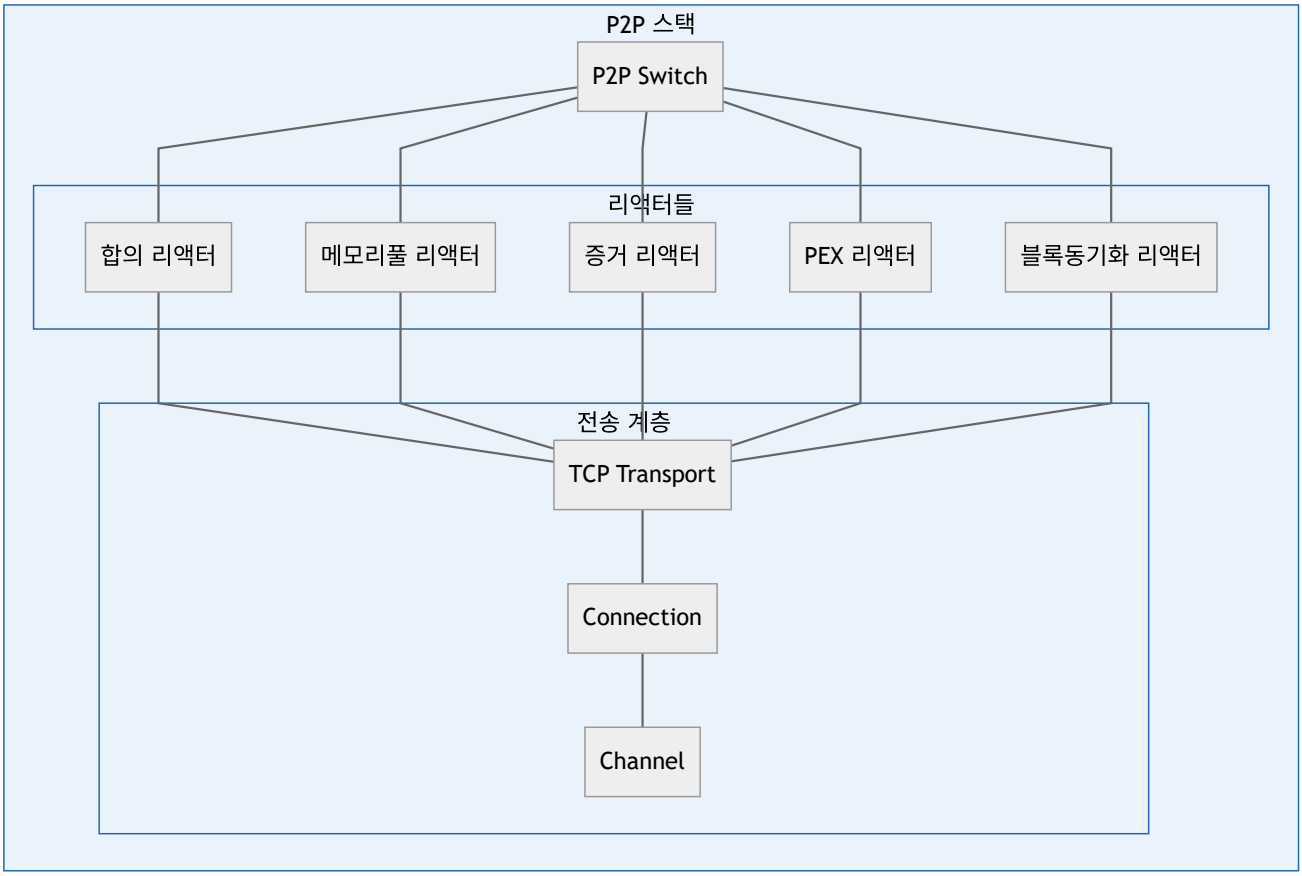
주요 책임:

- **블록 제안**: 제안자가 새로운 블록을 제안
- **투표 수집**: 검증자들의 투표를 수집하고 검증
- **합의 달성**: 2/3+ 합의를 통한 블록 확정
- **상태 전이**: 합의된 블록을 기반으로 상태 업데이트

3. P2P 네트워킹

분산 네트워크에서 노드 간 통신을 담당하는 레이어입니다.

아키텍처:



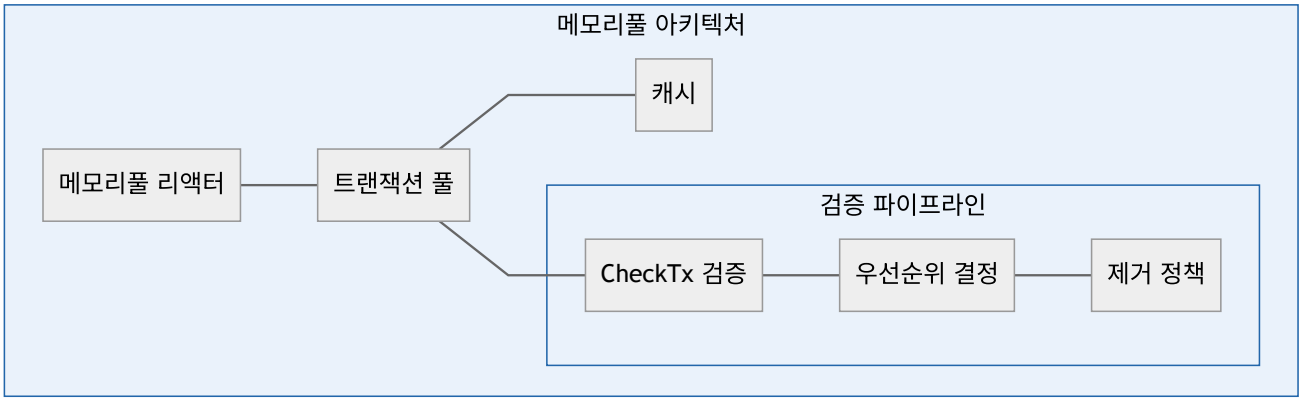
주요 기능:

- **피어 관리**: 피어 발견, 연결, 해제 관리
- **메시지 라우팅**: 다양한 프로토콜 메시지의 효율적 전달
- **네트워크 보안**: 인증 및 암호화된 통신
- **대역폭 최적화**: 중복 전송 방지 및 효율적 전파

4. 메모리풀 (Mempool)

아직 블록에 포함되지 않은 트랜잭션들을 관리하는 컴포넌트입니다.

구조:



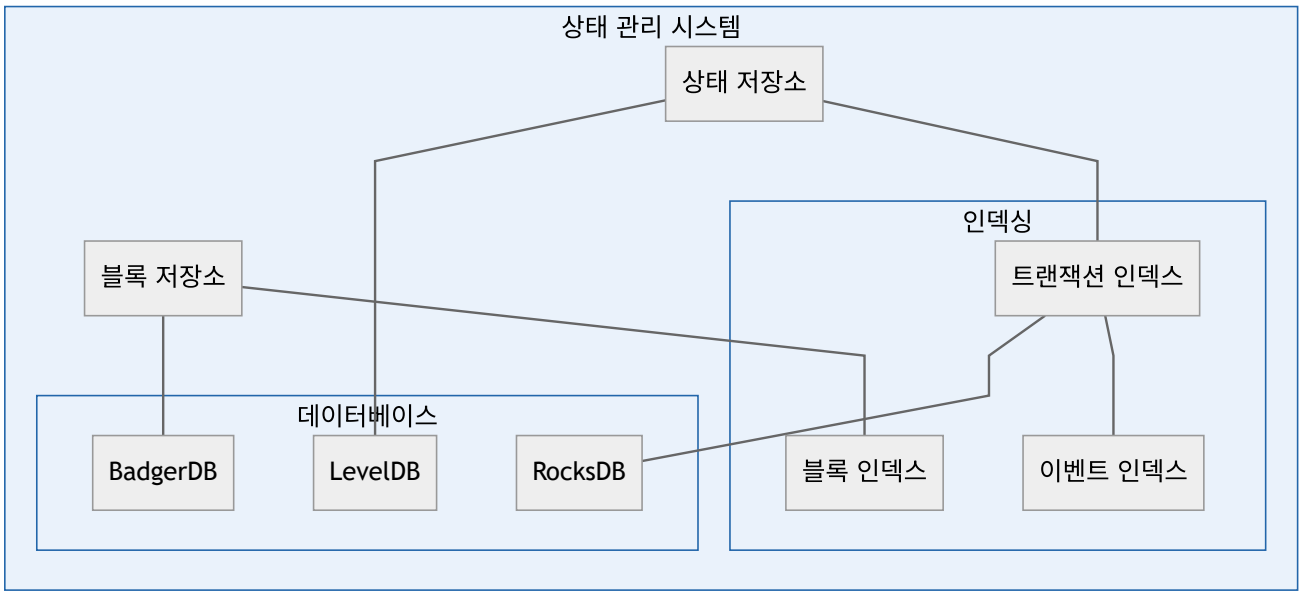
주요 기능:

- **트랜잭션 검증**: ABCI CheckTx를 통한 유효성 검사
- **우선순위 관리**: 가스 가격 등을 기반으로 한 우선순위 결정
- **캐시 관리**: 중복 트랜잭션 방지 및 메모리 효율성
- **네트워크 전파**: 유효한 트랜잭션의 피어 간 전파

5. 상태 관리 (State Management)

블록체인 상태와 블록 데이터를 관리하는 컴포넌트들입니다.

구조:



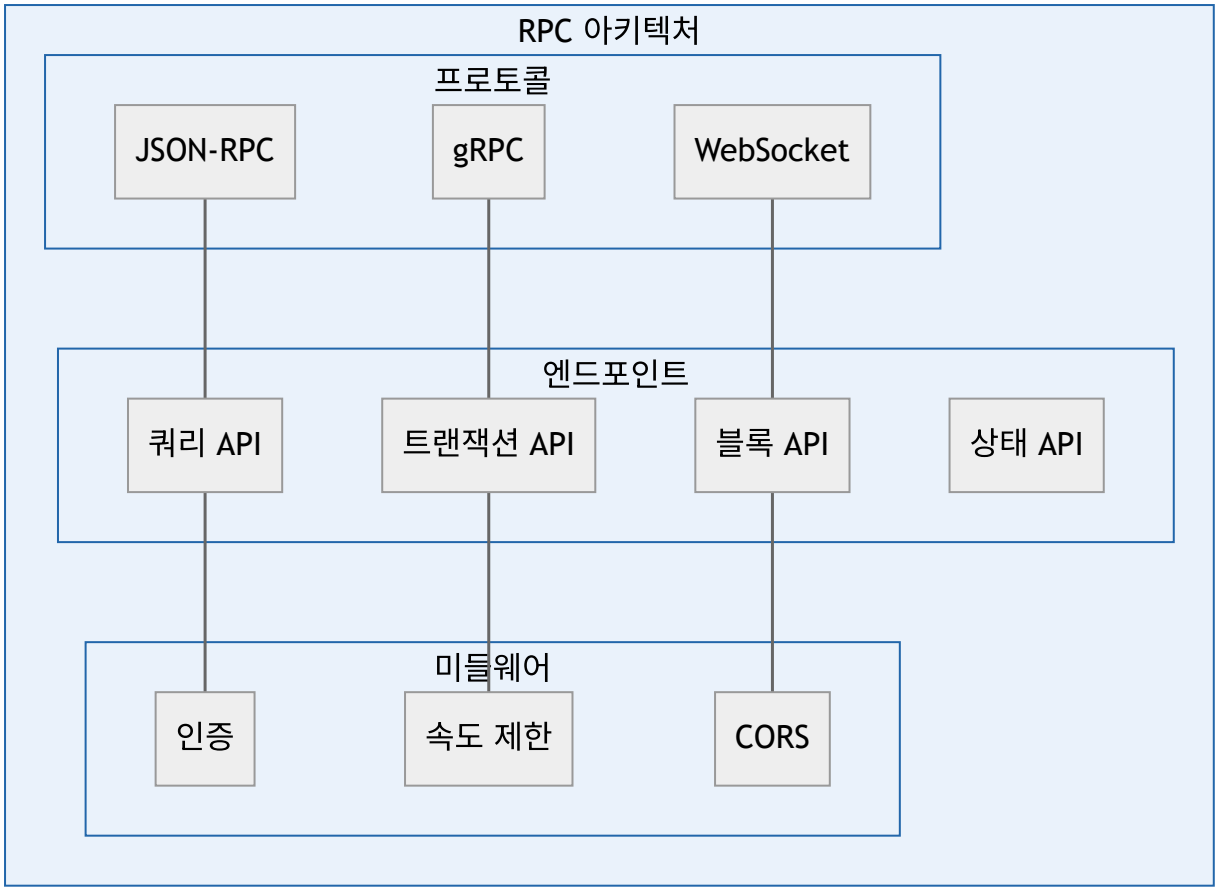
주요 기능:

- **상태 저장**: 애플리케이션 상태의 영구 저장
- **블록 저장**: 블록 데이터와 메타데이터 관리
- **인덱싱**: 빠른 조회를 위한 다양한 인덱스 제공
- **백업 및 복구**: 상태 스냅샷과 복구 메커니즘

6. RPC 서버

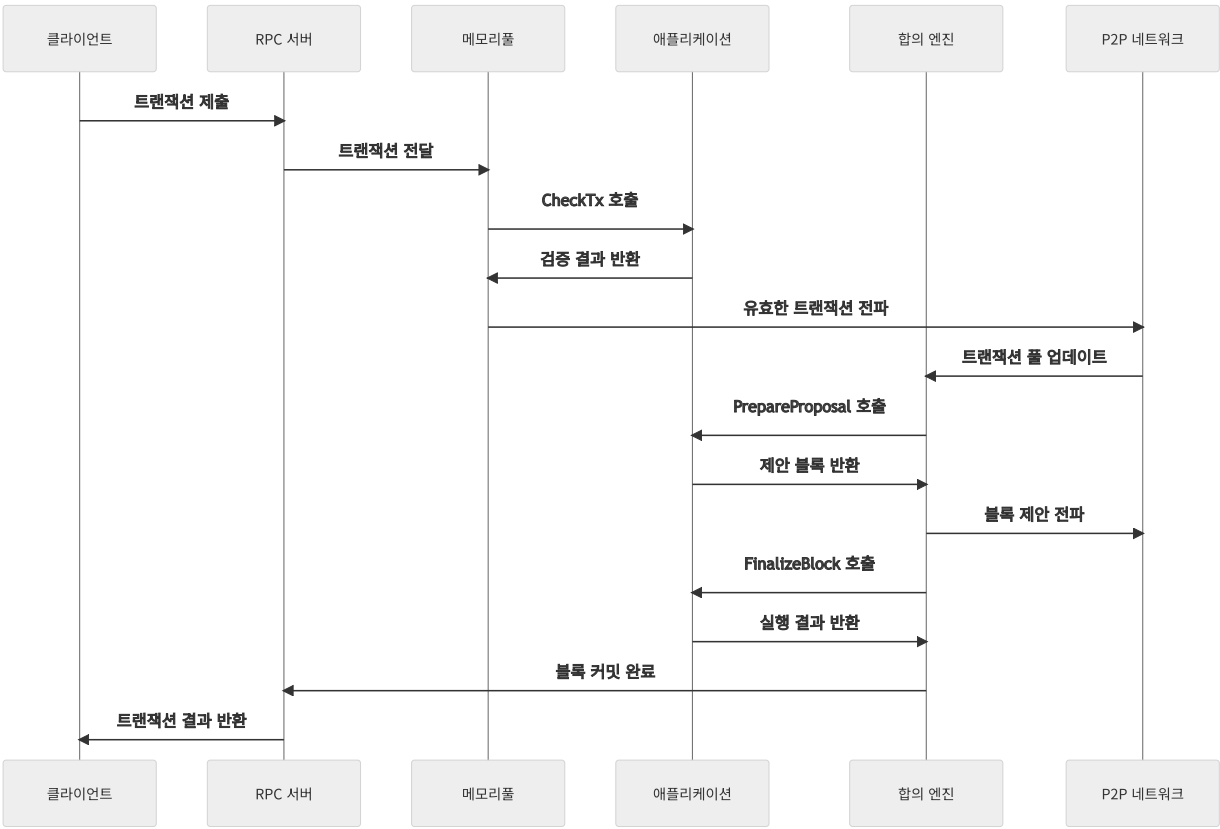
외부 클라이언트와의 인터페이스를 제공하는 서비스 레이어입니다.

구조:

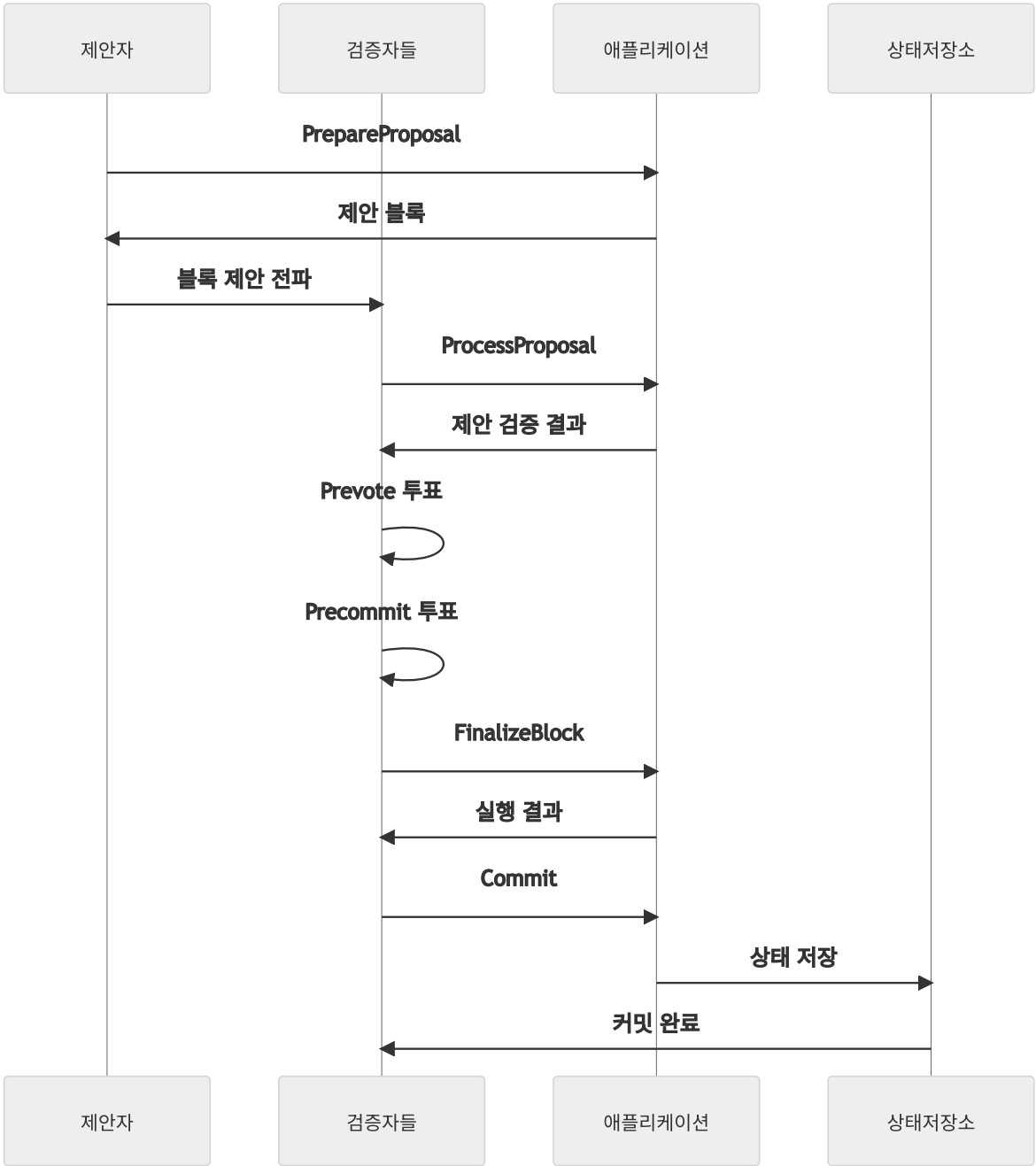


데이터 흐름 분석

1. 트랜잭션 처리 흐름

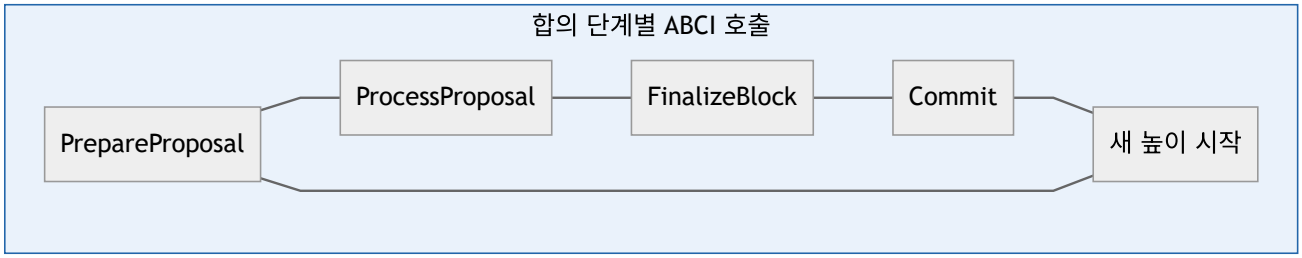

클라이언트
RPC 서버
메모리풀
애플리케이션
합의 엔진
P2P 네트워크
트랜잭션 제출
트랜잭션 전달
CheckTx 호출
검증 결과 반환
유효한 트랜잭션 전파
트랜잭션 풀 업데이트
PrepareProposal 호출
제안 블록 반환
블록 제안 전파
FinalizeBlock 호출
실행 결과 반환
블록 커밋 완료
트랜잭션 결과 반환
클라이언트
RPC 서버
메모리풀
애플리케이션
합의 엔진
P2P 네트워크

2. 블록 생성 및 합의 흐름

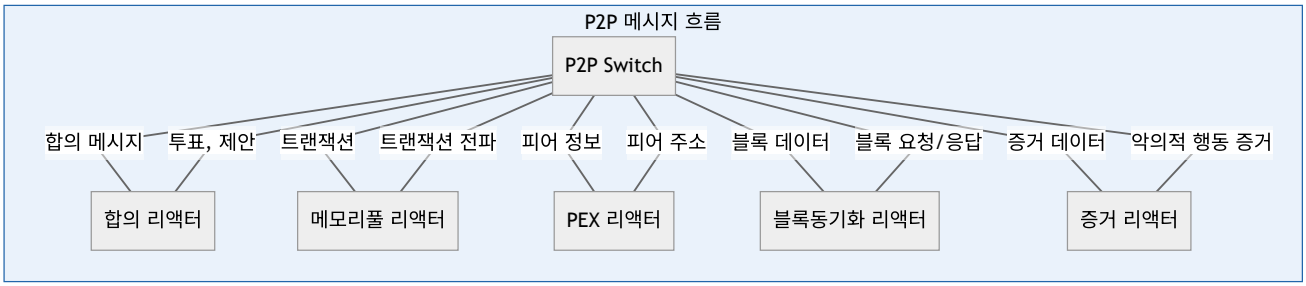


컴포넌트 간 상호작용

1. 합의 엔진과 ABCI



2. P2P와 리액터들



성능 및 확장성 고려사항

1. 병렬 처리

- **합의와 실행 분리**: 블록 합의와 트랜잭션 실행의 파이프라인 처리
- **P2P 메시지 처리**: 각 리액터별 독립적인 메시지 처리
- **상태 관리**: 읽기/쓰기 작업의 최적화

2. 메모리 관리

- **메모리풀 크기 제한**: 설정 가능한 메모리풀 크기 제한
- **캐시 전략**: LRU 캐시를 통한 효율적인 메모리 사용
- **가비지 컬렉션**: 주기적인 불필요한 데이터 정리

3. 네트워크 최적화

- **메시지 압축**: 대용량 데이터의 압축 전송
- **배치 처리**: 여러 메시지의 배치 전송
- **연결 풀링**: 효율적인 연결 관리

설정 및 튜닝

1. 핵심 설정 파라미터

```
# 합의 관련 설정
[consensus]
timeout_propose = "3s"
timeout_prevote = "1s"
timeout_precommit = "1s"
timeout_commit = "5s"

# P2P 네트워킹 설정
[p2p]
max_num_inbound_peers = 40
max_num_outbound_peers = 10
send_rate = 5120000
recv_rate = 5120000

# 메모리풀 설정
```

```
[mempool]

size = 5000

max_txs_bytes = 1073741824

cache_size = 10000



# 상태 동기화 설정

[statesync]

enable = false

rpc_servers = ""

trust_height = 0

trust_hash = ""
```

2. 성능 튜닝 가이드

- **하드웨어 요구사항**: CPU, 메모리, 디스크 I/O 최적화
- **네트워크 설정**: 대역폭과 지연시간 고려
- **데이터베이스 선택**: 사용 사례에 맞는 백엔드 선택

보안 고려사항

1. 네트워크 보안

- **피어 인증**: 공개키 기반 피어 인증
- **메시지 암호화**: 전송 중 데이터 암호화
- **DDoS 방어**: 속도 제한 및 연결 제한

2. 합의 보안

- **서명 검증**: 모든 투표와 제안의 서명 검증
- **비잔틴 장애 허용**: 1/3 미만의 악의적 노드 허용
- **타임스탬프 검증**: PBTS를 통한 시간 조작 방지

다음 단계

이 아키텍처 개요를 바탕으로 다음 문서들에서 각 컴포넌트를 더 자세히 살펴볼 수 있습니다:

- 핵심 개념 및 용어: 주요 용어와 개념 정리
- 합의 프로세스: Tendermint 합의 알고리즘 상세 분석
- ABCI 인터페이스: 애플리케이션 인터페이스 상세 설명
- P2P 네트워킹: 네트워킹 레이어 심화 학습

참고 자료

- CometBFT 사양서
- 아키텍처 결정 기록 (ADR)
- CometBFT 개발자 가이드

이 문서는 CometBFT v1.0.0을 기준으로 작성되었습니다.